Programming in Python
IDC 212
Lecture - 7

September 10, 2025

Contents

1. Exception

1.1 Pre-Defined Exception

1.2 User Defined Exception

2. File Handling

Exception

Types of Exceptions in Python

- In Python, exceptions are classified into two major types:
  1. Predefined Exceptions
  2. User Defined Exceptions

1) Predefined Exceptions

- Also known as built-in exceptions.
- Raised automatically by the Python runtime when something goes wrong.

Example 1: Division by zero

```python
print("Average:", 90 / 0)
# Raises ZeroDivisionError
```

Example 2: Invalid type conversion

```python
marks = int("ninety")
# Raises ValueError
```

2) User Defined Exceptions

- Also known as Custom or Programmatic Exceptions.
- Programmers create them to signal application-level problems.
- Use `raise` keyword to throw them explicitly.

User Defined Exception Example

Example: Custom exception in a university system

```python
class GradeTooHighError(Exception):
    pass

def assign_grade(score):
    if score > 100:
        raise GradeTooHighError("Score exceeds max limit!")
    print("Grade assigned:", score)

assign_grade(120)
```

Handling Custom Exception in Python

Define and Use Custom Exception

```python
class GradeTooHighError(Exception):
    pass  # Empty class still works for custom message

def assign_grade(score):
    if score > 100:
        raise GradeTooHighError("Score exceeds max limit!")
    print("Grade assigned:", score)

try:
    assign_grade(120)
except GradeTooHighError as e:
    print("Exception occurred:", e)
```

Output: `Exception occurred: Score exceeds max limit!`

Customized Exceptions: Cricket Team Registration

- Define exceptions for underfilled or overfilled teams.
- Use `raise` to trigger exceptions based on team size.
- Handle them using `try–except` blocks.

Defining Custom Exceptions

Define custom exception classes:

```
class IncompleteTeamError(Exception):
    def __init__(self, message):
        self.msg = message

class PlayerLimitExceededError(Exception):
    def __init__(self, message):
        self.msg = message
```

Using the Custom Exceptions

Registering cricket team players and handling exceptions:

```python
try:
    team_size = int(input("Enter number of players: "))

    if team_size < 11:
        raise IncompleteTeamError("Minimum 11 players required to form a cricket team!")
    elif team_size > 15:
        raise PlayerLimitExceededError("Maximum allowed players are 15!")

    else:
        print("Team registered successfully. Good luck!")
```

Handling the Exceptions

Handling both underfilled and overfilled team exceptions:

```
except IncompleteTeamError as e:
    print("Registration Error:", e.msg)

except PlayerLimitExceededError as e:
    print("Registration Error:", e.msg)
```

Key Points

- Use `Exception` as the base class for all custom errors.
- The `raise` keyword is used to explicitly trigger these exceptions.
- Always handle custom exceptions using `try–except` blocks.
- Keep exception messages user-friendly and specific.

Note: `raise` is best suited for custom exceptions, not predefined ones like `ZeroDivisionError`.

File Handling

File Open Modes in Python (Text Files)

- `"r"` **(Read)**: Opens existing file for reading.
  - File must exist; otherwise, `FileNotFoundError`.
  - File pointer is placed at the **beginning**.
- `"w"` **(Write)**: Opens file for writing.
  - If file exists, its contents are erased.
  - If not, it creates a new file.
  - File pointer is at the **start**.
- `"a"` **(Append)**: Opens file to append data.
  - File is created if it doesn't exist.
  - File pointer is at the **end**; data is added after current content.

Modes Supporting Read + Write

- `"r+"` **(Read and Write)**:
  - File must exist; no truncation.
  - File pointer starts at the **beginning**.
- `"w+"` **(Write and Read)**:
  - Truncates if file exists; creates new otherwise.
  - Pointer is at the **start**; file is empty initially.
- `"a+"` **(Append and Read)**:
  - Creates file if missing.
  - Pointer is at the **end** for write, but can read anywhere using `seek()`.

Exclusive Creation Mode and Binary Modes

- `"x"` **(Exclusive Creation)**:
  - Creates a file for writing.
  - Raises error if file exists.
  - File pointer is at the **start**.
- Add `"b"` to any mode to handle binary files (e.g., `"rb"`, `"wb"`).
  - Used for non-text data like images, videos, PDFs.
  - File pointer rules are the same as their text counterparts.

Example: Pointer Positions

```python
# Example for 'a+' mode
f = open("log.txt", "a+")
f.write("New log entry\n")
f.seek(0)                # move pointer to start to read
print(f.read())          # read entire file
f.close()
```

- a+ mode appends at the end, but allows reading from any point using `seek()`.

Closing a File

After completing file operations like writing or reading, it's essential to close the file to:

- Save data to disk properly
- Release system resources
- Prevent data corruption

We use the `close()` method:

```
f = open("sports_log.txt", "w")
f.close()
```

File Object Properties

Once a file is opened, Python returns a file object. It has several useful attributes:

- `.name` – File name
- `.mode` – Mode used to open
- `.closed` – Checks if file is closed
- `.readable()` – Checks read permission
- `.writable()` – Checks write permission

Example: Accessing File Properties

```python
f = open("sports_log.txt", 'w')

print("Opened File:", f.name)
print("Access Mode:", f.mode)
print("Can Read?:", f.readable())
print("Can Write?:", f.writable())
print("Is Closed (Before):", f.closed)

f.close()

print("Is Closed (After):", f.closed)
```

Output Snapshot

```
Opened File: sports_log.txt
Access Mode: w
Can Read?: False
Can Write?: True
Is Closed (Before): False
Is Closed (After): True
```

Writing Text to Files in Python

We can store textual data into files using:

1. `write(str)` – Writes a single string.
2. `writelines(list)` – Writes a list of strings.

Tip: Include newline characters `("\n")` to separate lines properly.

Example: Writing Technical Book Titles

```
f = open("cs_books.txt", "w")
f.write("Introduction to Algorithms\n")
f.write("Structure and Interpretation of Computer Programs\n")
f.write("Clean Code\n")
print("Books saved successfully.")
f.close()
```

Result in cs_books.txt:

- Introduction to Algorithms
- Structure and Interpretation of Computer Programs
- Clean Code

Appending to the File (No Overwrite)

```
f = open("cs_books.txt", "a")
f.write("Artificial Intelligence: A Modern Approach\n")
f.close()
```

This appends a line to the existing file without removing previous content.

Using writelines() for Authors

```python
f = open("cs_authors.txt", "w")
authors = [
    "Thomas H. Cormen\n",
    "Harold Abelson\n",
    "Robert C. Martin\n",
    "Stuart Russell\n"
]
f.writelines(authors)
print("Authors written successfully.")
f.close()
```

Result in cs_authors.txt:

- Thomas H. Cormen
- Harold Abelson
- Robert C. Martin
- Stuart Russell

Key Reminders

- `"w"` mode creates/overwrites files.
- `"a"` mode appends to files without erasing old content.
- Always close files with `f.close()`.
- Without `"\n"`, all data is written in a single line.

Reading Data from Text Files in Python

We can read character data from text files using the following methods:

- `read()` – Reads the entire contents of the file as a single string.
- `read(n)` – Reads up to `n` characters from the file.
- `readline()` – Reads a single line from the file.
- `readlines()` – Reads all lines and returns them as a list.

These methods are useful for processing structured text such as logs, CSVs, and documents.

Example 1: Read Entire Content

```python
f = open("books.txt", 'r')
data = f.read()
print(data)
f.close()
```

Sample Output:

- Introduction to Algorithms
- Clean Code
- AI: A Modern Approach
- Python for Data Analysis

Example 2: Read First 25 Characters

```python
f = open("books.txt", 'r')
data = f.read(25)
print(data)
f.close()
```

Sample Output: `Introduction to Algorithm`

Example 3: Read Line-by-Line

```python
f = open("books.txt", 'r')

line1 = f.readline()
print("Book 1:", line1.strip())

line2 = f.readline()
print("Book 2:", line2.strip())

line3 = f.readline()
print("Book 3:", line3.strip())

f.close()
```

Output:

- Book 1: Introduction to Algorithms
- Book 2: Clean Code
- Book 3: AI: A Modern Approach

Example 4: Read All Lines as List

```python
f = open("books.txt", 'r')
lines = f.readlines()

for i, line in enumerate(lines):
    print(f"Book {i+1}:", line.strip())

f.close()
```

Output:

- Book 1: Introduction to Algorithms
- Book 2: Clean Code
- Book 3: AI: A Modern Approach
- Book 4: Python for Data Analysis

The `with` Statement in File Handling

- The `with` statement in Python provides a way to manage resources like files.
- It automatically handles opening and closing the file, even if errors occur.
- Syntax:

```python
with open("cs_books.txt", "w") as f:
    f.write("Artificial Intelligence\n")
    f.write("Computer Networks\n")
    f.write("Automata Theory\n")
    print("Is File Closed (inside block):", f.closed)
print("Is File Closed (outside block):", f.closed)
```

Output:

- Inside block: False
- Outside block: True

The `tell()` Method: Tracking File Pointer

- The `tell()` method returns the current position (in bytes) of the file cursor.
- This is useful when reading or writing at specific file positions.
- Example:

```python
f = open("cs_books.txt", "r")
print(f.tell())        # Cursor at beginning: 0
print(f.read(11))      # Reads: "Artificial "
print(f.tell())        # Now cursor is at 11
print(f.read(9))       # Reads: "Intellig"
print(f.tell())        # Now cursor is at 20
f.close()
```

Note: Positions may differ with encoding; here, we assume simple ASCII text.

The seek() Method: Moving File Pointer

- The `seek(offset)` method repositions the file pointer to a specified byte.
- You can combine it with `tell()` for random access within the file.

```
f = open("cs_books.txt", "r")
print(f.read(10))      # Reads: "Artificial"
f.seek(0)              # Reset to beginning
print(f.read(9))       # Reads: "Artificia"
f.seek(12)             # Jump to "Computer"
print(f.read(8))       # Reads: "Computer"
f.close()
```

Note: `seek()` is especially useful for skipping unwanted portions of the file.

`seek()` Method in Python

- The `seek()` method is used to move the file pointer to a specific position.
- Syntax: `f.seek(offset, from_where)`
  - **offset**: number of bytes to move the pointer
  - **from_where** (optional):
    - 0 ⇒ from start of file (default)
    - 1 ⇒ from current position (not supported in text mode)
    - 2 ⇒ from end of file (not supported in text mode)
- Use case: modify a file by jumping to a specific position before writing.

Example: Modifying Text Using `seek()`

```
data = "Don Knuth wrote many books."
f = open("authors.txt", "w")
f.write(data)
f.close()
```

Now we reopen the file in `r+` mode to both read and modify contents.

```
with open("authors.txt", "r+") as f:
    print("Before Modification:")
    print(f.read())
    print("Cursor Position:", f.tell())
```

Modifying the File Using seek()

```
    f.seek(22)  # Move to start of word 'books'
    print("New Cursor Position:", f.tell())
    f.write("texts")  # Overwrite 'books' with 'texts'
    f.seek(0)  # Go back to beginning
    print("After Modification:")
    print(f.read())
```

Output:

- Before Modification: Don Knuth wrote many books.
- Cursor Position: 30
- New Cursor Position: 22
- After Modification: Don Knuth wrote many texts.

Summary: `seek()` vs `tell()`

- `tell()`:
  - Returns the current position of the file pointer (starting from 0).
- `seek(offset)`:
  - Moves the file pointer to the specified byte position.
  - Only 0 is valid for `from_where` in text mode in Python 3.
- Ideal for modifying parts of a file without rewriting everything.

Note: These functions are mostly used in scenarios requiring random access.

Step 1: File Name Input and Check

Objective: Read file name from the user and check if the file exists using the `os.path.isfile()` method.

```python
import os, sys

filename = input("Enter file name: ")

if os.path.isfile(filename):
    print("File found:", filename)
    f = open(filename, "r")
else:
    print("File not found:", filename)
    sys.exit(0)
```

Step 2: Initialize Counters

Initialize variables to keep track of:

- Line count
- Word count
- Character count

```
line_count = 0
word_count = 0
char_count = 0
```

Step 3: Loop Through the File

Read the file line-by-line and update all counters.

```
for line in f:
    line_count += 1
    char_count += len(line)
    words = line.split()
    word_count += len(words)
```

Step 4: Print the Results and Close File

Display the final counts and close the file to release resources.

```
print("Total Lines:", line_count)
print("Total Words:", word_count)
print("Total Characters:", char_count)

f.close()
```

Sample Output 1: File Does Not Exist

Console Output

```
Enter file name: notes.txt
File not found: notes.txt
```

Sample Output 2: File Exists

Console Output

```
Enter file name: sample.txt
File found: sample.txt
Total Lines: 4
Total Words: 18
Total Characters: 102
```

Sample Input File: sample.txt

Contents of sample.txt

```
Python is powerful
It is widely used in AI
File handling is important
Be consistent while coding
```

Question 1: Read from a File Safely

Problem:

- Ask the user to enter a filename.
- If the file exists, read and print each line with line numbers.
- If the file doesn't exist, print an error message.
- Also handle the case where the file is empty.

Solution to Question 1

```python
try:
    filename = input("Enter filename: ")
    with open(filename, 'r') as f:
        lines = f.readlines()
        if not lines:
            print("File is empty.")
        else:
            for i, line in enumerate(lines, start=1):
                print(f"Line {i}: {line.strip()}")
except FileNotFoundError:
    print("The file does not exist.")
except IOError:
    print("An I/O error occurred.")
```

Question 2: Write User Input to File

Problem:

- Ask user to input multiple lines of text.
- Stop input when user types `STOP`.
- Save lines into a file `output.txt`.
- Handle permission and unexpected exceptions.

Solution to Question 2: Part 1

```
try:
    print("Type lines of text (type 'STOP' to finish):")
    lines = []
    while True:
        line = input()
        if line.upper() == 'STOP':
            break
        lines.append(line + '\n')
```

Solution to Question 2: Part 2

```
    with open("output.txt", "w") as file:
        file.writelines(lines)
    print("Lines saved to output.txt")

except PermissionError:
    print("Permission denied: can't write file.")
except Exception as e:
    print("Unexpected error:", e)
```

Summary

- Use `try-except` for robust file access.
- Prefer `with open()` for safety.
- Always handle edge cases: file not found, empty file, permission error.

Thank You!

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has added to receive it.
If you rerun the document (without altering it) this surplus page will go away, becau LaTeX now knows how many pages to expect for this document.